

PRÁCTICA GUIADA – ETAPA 7

PROYECTO "SERVICIOS TÉCNICOS"

Módulo de Técnicos – CRUD completo e integración con React

1. Propósito

Construir el módulo Técnicos reutilizando la arquitectura de las etapas anteriores. Se creará una API REST CRUD en Flask, se probará con Thunder Client y se integrará una pantalla protegida en React.

2. Objetivos

- Identificar la estructura real de la tabla mediante DESCRIBE.
- Implementar GET, POST, PUT y DELETE.
- Usar consultas SQL parametrizadas.
- Probar la API con Thunder Client.
- Crear servicios de API en React.
- Crear la interfaz CRUD de Técnicos.
- Agregar la ruta protegida /tecnicos.
- Sincronizar los cambios con GitHub.

3. Requisitos previos

- Etapa 6 terminada.
- Login, Dashboard y RutaProtegida funcionando.
- Flask en http://127.0.0.1:5000.
- React en http://localhost:5173/.
- MySQL y base tecnico disponibles.
- Thunder Client instalado.

4. Revisar la tabla de Técnicos

En MySQL ejecuta:

```
USE tecnico;
SHOW TABLES;
DESCRIBE tecnico;
```

Si la tabla tiene otro nombre, usa exactamente el nombre mostrado. Los ejemplos de esta práctica suponen id, nombre y especialidad. Adapta todos los campos a tu estructura real.



5. Iniciar los servidores

```
cd backend
venv\Scripts\activate
python -m app.main

cd frontend
npm run dev
```

6. Crear la ruta Flask

Crea backend/app/routes/tecnicos.py:

```
from flask import Blueprint, jsonify, request
from config.conexion import conectar

tecnicos_bp = Blueprint("tecnicos", __name__)
```

7. GET – listar técnicos

```
@tecnicos_bp.route("/api/tecnicos", methods=["GET"])
def obtener_tecnicos():
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor(dictionary=True)
        cursor.execute("SELECT * FROM tecnico")
        return jsonify({"success": True, "tecnicos": cursor.fetchall()}), 200
    except Exception as e:
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

8. POST – crear técnico

```
@tecnicos_bp.route("/api/tecnicos", methods=["POST"])
def crear_tecnico():
    datos = request.get_json(silent=True)
    if not isinstance(datos, dict):
        return jsonify({"success": False, "message": "JSON válido requerido"}), 400

    nombre = datos.get("nombre")
    especialidad = datos.get("especialidad")
    if not nombre or not especialidad:
        return jsonify({"success": False, "message": "Nombre y especialidad son obligatorios"}), 400

    conexion = conectar()
    if conexion is None:
```




```
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute(
            "INSERT INTO tecnico (nombre, especialidad) VALUES (%s, %s)",
            (nombre.strip(), especialidad.strip())
        )
        conexion.commit()
        return jsonify({"success": True, "message": "Técnico creado", "id": cursor.lastrowid}), 201
    except Exception as e:
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

9. PUT – actualizar técnico

```
@tecnicos_bp.route("/api/tecnicos/<int:tecnico_id>", methods=["PUT"])
def actualizar_tecnico(tecnico_id):
    datos = request.get_json(silent=True)
    if not isinstance(datos, dict):
        return jsonify({"success": False, "message": "JSON válido requerido"}), 400

    nombre = datos.get("nombre")
    especialidad = datos.get("especialidad")
    if not nombre or not especialidad:
        return jsonify({"success": False, "message": "Campos obligatorios"}), 400

    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute(
            "UPDATE tecnico SET nombre=%s, especialidad=%s WHERE id=%s",
            (nombre.strip(), especialidad.strip(), tecnico_id)
        )
        conexion.commit()
        if cursor.rowcount == 0:
            return jsonify({"success": False, "message": "Técnico no encontrado"}), 404
        return jsonify({"success": True, "message": "Técnico actualizado"}), 200
    except Exception as e:
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
```




```
        if cursor: cursor.close()
        conexion.close()
```

10. DELETE – eliminar técnico

```
@tecnicos_bp.route("/api/tecnicos/<int:tecnico_id>", methods=["DELETE"])
def eliminar_tecnico(tecnico_id):
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute("DELETE FROM tecnico WHERE id=%s", (tecnico_id,))
        conexion.commit()
        if cursor.rowcount == 0:
            return jsonify({"success": False, "message": "Técnico no encontrado"}), 404
        return jsonify({"success": True, "message": "Técnico eliminado"}), 200
    except Exception as e:
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

Usa DELETE solamente sobre un registro creado para la práctica. Las relaciones de MySQL pueden impedir eliminar registros relacionados.

11. Registrar el Blueprint

En backend/app/main.py agrega:

```
from app.routes.tecnicos import tecnicos_bp
app.register_blueprint(tecnicos_bp)
```

Guarda y reinicia Flask con Ctrl+C y luego python -m app.main.

12. Probar GET con Thunder Client

1. New Request → GET.
2. URL: http://127.0.0.1:5000/api/tecnicos.
3. Send y verifica 200 OK.

```
{"success":true,"tecnicos":[]}
```

Si hay registros, aparecerán dentro del arreglo.

13. Probar POST

4. POST a http://127.0.0.1:5000/api/tecnicos.



5. Body → JSON.
6. Enviar un registro de prueba:

```
{"nombre":"Técnico de prueba","especialidad":"Computadoras"}
```

Esperado: 201 Created y un ID. Si tu tabla tiene más campos obligatorios, agrégalos.

14. Probar PUT

7. PUT a http://127.0.0.1:5000/api/tecnicos/ID.
8. Sustituye ID por el ID obtenido.
9. Body → JSON:

```
{"nombre":"Técnico actualizado","especialidad":"Redes"}
```

Esperado: 200 OK.

15. Probar DELETE

10. DELETE a http://127.0.0.1:5000/api/tecnicos/ID.
11. Usa el registro de prueba.
12. Send y después GET para comprobar que desapareció.

Esperado: 200 OK.

16. Servicios React

En frontend/src/services/api.js agrega:

```
const API_TECNICOS = "http://127.0.0.1:5000/api/tecnicos";

export async function obtenerTecnicos() {
  const r = await fetch(API_TECNICOS);
  return { ok: r.ok, datos: await r.json() };
}

export async function crearTecnico(tecnico) {
  const r = await fetch(API_TECNICOS, {
    method: "POST",
    headers: {"Content-Type":"application/json"},
    body: JSON.stringify(tecnico)
  });
  return { ok: r.ok, datos: await r.json() };
}

export async function actualizarTecnico(id, tecnico) {
  const r = await fetch(`${API_TECNICOS}/${id}`, {
    method: "PUT",
    headers: {"Content-Type":"application/json"},
```




```
    body: JSON.stringify(tecnico)
  });
  return { ok: r.ok, datos: await r.json() };
}

export async function eliminarTecnico(id) {
  const r = await fetch(`${API_TECNICOS}/${id}`, {method:"DELETE"});
  return { ok: r.ok, datos: await r.json() };
}
```

17. Crear Tecnicos.jsx

Crea frontend/src/pages/Tecnicos.jsx. Utiliza este componente base y adapta los campos a tu tabla:

```
import { useEffect, useState } from "react";
import {
  obtenerTecnicos, crearTecnico,
  actualizarTecnico, eliminarTecnico
} from "../services/api";

function Tecnicos() {
  const [tecnicos,setTecnicos]=useState([]);
  const [nombre,setNombre]=useState("");
  const [especialidad,setEspecialidad]=useState("");
  const [editandoId,setEditandoId]=useState(null);
  const [mensaje,setMensaje]=useState("");

  const cargarTecnicos=async()=>{
    const r=await obtenerTecnicos();
    if(r.ok && r.datos.success) setTecnicos(r.datos.tecnicos);
    else setMensaje(r.datos.message || "Error al cargar");
  };

  useEffect(()=>{ cargarTecnicos(); },[]);

  const guardar=async(e)=>{
    e.preventDefault();
    if(!nombre.trim() || !especialidad.trim()){
      setMensaje("Todos los campos son obligatorios"); return;
    }
    const datos={nombre,especialidad};
    const r=editandoId
      ? await actualizarTecnico(editandoId,datos)
      : await crearTecnico(datos);
    setMensaje(r.datos.message || "Operación realizada");
    if(r.ok){
      setNombre(""); setEspecialidad(""); setEditandoId(null);
      await cargarTecnicos();
    }
```




```
};

const editar=(tecnico)=>{
  setEditandoId(tecnico.id);
  setNombre(tecnico.nombre);
  setEspecialidad(tecnico.especialidad);
};

const eliminar=async(id)=>{
  if(!window.confirm("¿Deseas eliminar este técnico?")) return;
  const r=await eliminarTecnico(id);
  setMensaje(r.datos.message || "Operación realizada");
  if(r.ok) await cargarTecnicos();
};

return (
  <div className="dashboard">
    <h1>Técnicos</h1>
    <form onSubmit={guardar}>
      <input value={nombre}
        onChange={e=>setNombre(e.target.value)}
        placeholder="Nombre"/>
      <input value={especialidad}
        onChange={e=>setEspecialidad(e.target.value)}
        placeholder="Especialidad"/>
      <button type="submit">
        {editandoId ? "Actualizar" : "Registrar"}
      </button>
      {editandoId && <button type="button"
        onClick={()=>{setEditandoId(null);setNombre("");setEspecialidad("");}}>
        Cancelar
      </button>}
    </form>

    {mensaje && <p>{mensaje}</p>}

    <table>
      <thead><tr>
        <th>ID</th><th>Nombre</th><th>Especialidad</th><th>Acciones</th>
      </tr></thead>
      <tbody>
        {tecnicos.map(tecnico=>
          <tr key={tecnico.id}>
            <td>{tecnico.id}</td>
            <td>{tecnico.nombre}</td>
            <td>{tecnico.especialidad}</td>
            <td>
              <button onClick={()=>editar(tecnico)}>Editar</button>
              <button onClick={()=>eliminar(tecnico.id)}>Eliminar</button>
```




```
              </td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  );
}
export default Tecnicos;
```

18. Agregar ruta protegida

En frontend/src/App.jsx:

```
import Tecnicos from "./pages/Tecnicos";

<Route path="/tecnicos"
  element={<RutaProtegida><Tecnicos /></RutaProtegida>}
/>
```

En Dashboard.jsx agrega:

```
<Link to="/tecnicos">Técnicos</Link>
```

19. Prueba integral

13. Inicia MySQL, Flask y Vite.
14. Inicia sesión.
15. Entra a Técnicos.
16. Registra un técnico de prueba.
17. Comprueba el listado.
18. Edita el técnico.
19. Comprueba el cambio.
20. Elimínalo.
21. Cierra sesión e intenta abrir /tecnicos directamente.

Resultado esperado: CRUD funcional y ruta protegida.

20. Errores frecuentes

Problema	Revisión
404	URL, Blueprint y register_blueprint.
405	Método HTTP y methods del decorador.
400	Body JSON y campos obligatorios.
500	Tabla, columnas, permisos y restricciones.
Failed to fetch	Flask y CORS.
No aparece	Respuesta GET y nombres de campos React.



No abre /tecnicos	App.jsx y RutaProtegida.

21. Evidencias

- SHOW TABLES y DESCRIBE.
- tecnicos.py.
- main.py con Blueprint.
- GET, POST, PUT y DELETE en Thunder Client.
- Prueba 400 y prueba 404.
- Pantalla Técnicos funcionando.
- Registro, edición y eliminación desde React.
- Ruta protegida.
- Repositorio GitHub actualizado.

22. Registro de pruebas

Caso	Método/ruta	Esperado	Resultado	Evidencia
Listar	GET /api/tecnicos	200		
Crear	POST /api/tecnicos	201		
Editar	PUT /api/tecnicos/{id}	200		
Eliminar	DELETE /api/tecnicos/{id}	200		
Validar	POST incompleto	400		
No existe	DELETE /api/tecnicos/999999	404		

23. Preguntas de análisis

22. ¿Por qué se revisa DESCRIBE antes de programar?

Respuesta: _____

23. ¿Qué ventaja tiene separar tecnicos.py de clientes.py?

Respuesta: _____

24. ¿Qué diferencia existe entre POST, PUT y DELETE?

Respuesta: _____

25. ¿Por qué se usan consultas parametrizadas?

Respuesta: _____



26. ¿Para qué sirven commit y rollback?

Respuesta: _____

27. ¿Por qué React no se conecta directamente a MySQL?

Respuesta: _____

28. ¿Qué función cumple RutaProtegida?

Respuesta: _____

24. Git y GitHub

```
git status
git add .
git commit -m "Agrega CRUD de tecnicos"
git push
```

Comprueba en GitHub que el commit y los archivos estén sincronizados.

27. Criterio de finalización

La Etapa 7 termina cuando el estudiante puede registrar, consultar, modificar y eliminar técnicos desde React, demostrar las operaciones en Thunder Client, mantener protegida la ruta /tecnicos y sincronizar el proyecto con GitHub.